Protocol Audit Report

Version 1.0

Yul Puma

April 27, 2026

Protocol Audit Report

Yul Puma

April 27, 2025

Prepared by: Yul Puma

Lead Security Researcher: - Yul Puma

Table of Contents

- Informational
  - [I-1] The `PasswordStore::getPassword` natspec indicates a parameter that does not exist, causing the natspec to be incorrect.
- Likelihood & Impact:

Protocol Summary

PasswordStore is a protocol which allows a user to be able to set a password, that is only visible to them. They should also be able to retrieve and update it at any time.

Disclaimer

The YOUR_NAME_HERE team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond the following commit hash: Commit Hash:

```
2e8f81e263b3a9d18fab4fb5c46805ffc10a9990
```

Scope

```
./src/
#-- PasswordStore.sol
```

Roles

- Owner: The user who can set the password and read the password.
- Outsiders: No one else should be able to set or read the password.

Executive Summary

We found some very apparent bugs and unintended functionality after completing the audit. The 3 findings, each categorized based on their severity and impact, can be found in the `findings.md` file for more information. We spent around 2 hours with the Lead Security Researcher using various tools/methods to discover the impact, create a POC, and formulate a recommended mitigation strategy for each corresponding finding.

Issues found

Severity	Number of issues found
High	2
Medium	0
Low	0
Info	1
Total	3

Findings

High

[H-1] Storing the password on-chain makes it visible to anyone, and no longer private

Description: All data stored on-chain is visible to anyone, and can be read directly from the blockchain. The `PasswordStore::s_password` variable is intended to be private variable and only accessed through the `PasswordStore::getPassword` function, which is intended to be only called by the owner of the contract.

We show one such method of reading any off-chain below. **Impact:** Anyone can read the private password, severely breaking the functionality of the protocol.

Proof of Concept:

The below test case shows how anyone can read the password directly from the blockchain.

1. Create a locally running chain

```
anvil
```

2. Deploy smart contract to local chain

```
make deploy
```

3. Run the storage tool

`s_password` is stored in storage slot 1 for PasswordStore contract.

```
cast storage ADDRESS_HERE 1 --rpc-url http://127.0.0.1:8545
```

You will get an output that looks like this: `0x6d7950617373776f726400000000000000000000000000000000`

If you parse the hex to a string

```
cast parse-bytes32-string 0
    x6d7950617373776f726400000000000000000000000000000000000000000014
```

The output shown will be `myPassword`

Recommended Mitigation: Due to this, the protocol should be rethought. A possible solution is to encrypt the password off-chain, and then store the encrypted password on-chain. This would require the user/owner of the contract to remember another password off-chain to decrypt the password.

This option would also require you to remove the view function as you wouldn't want the user to accidentally send a transaction with the password that decrypts your password.

Likelihood & Impact:

- Impact: HIGH
- Likelihood: HIGH
- Severity: HIGH

[H-2] `PasswordStore::setPassword` has no access controls, meaning a non-owner could change the password.

Description: The `PasswordStore::setPassword` function is set to be an `external` function however, the natspec of the function and the overall purpose of the smart contract is that `This function allows only the owner to set a new password.`

```
1      function setPassword(string memory newPassword) external {
2          s_password = newPassword;
3          emit SetNetPassword();
4      }
```

Impact: Any user (owner & non-owner) can update the password of the contract, severely breaking the contract's intended purpose.

Proof of Concept: Add the following to the `PasswordStore.t.sol` test file.

Code

```
1      function test_anyone_can_set_password(address randomAddress) public
           {
2          vm.assume(randomAddress != owner);
3          vm.prank(randomAddress);
4          string memory expectedPassword = "myNewPassword";
5          passwordStore.setPassword(expectedPassword);
6
7          vm.prank(owner);
8          string memory actualPassword = passwordStore.getPassword();
9          assertEq(actualPassword, expectedPassword);
10     }
```

Recommended Mitigation: Add an access control conditional to the `setPassword` function.

```
1      if (msg.sender != owner)
2          revert PasswordStore__NotOwner();
```

Likelihood & Impact:

- Impact: HIGH
- Likelihood: HIGH
- Severity: HIGH

Informational

[I-1] The `PasswordStore::getPassword` natspec indicates a parameter that does not exist, causing the natspec to be incorrect.

Description:

```
    /*
     * @notice This allows only the owner to retrieve the password.
     // @audit there is no newPassword parameter.
     * @param newPassword The new password to set.
     */
    function getPassword() external view returns (string memory) {
        if (msg.sender != s_owner) {
            revert PasswordStore__NotOwner();
        }
        return s_password;
    }
```

The `PasswordStore::getPassword` function signature is `getPassword()` which the natspec says it should be `getPassword(string)`.

Impact: The natspec is incorrect.

Recommended Mitigation: Remove the incorrect natspec line.

```
-   * @param newPassword The new password to set.
```

Likelihood & Impact:

- Impact: None
- Likelihood: HIGH?
- Severity: Informational/Gas/Non-critical